

废土挂机编程游戏设计拆解文档

——从《编程农场》的机制启发到废土机器人 RPG 自动化系统

一、核心定位

这款游戏的表层包装是：

捡到机器人
→ 垃圾场拾荒
→ 改装机器人
→ 角斗场战斗
→ 迷宫探险

但它真正的核心体验应该是：

玩家不是在“操控一个机器人”，
而是在逐步为一个捡来的破机器人编写操作系统。

玩家最初只是让机器人走一步、捡一块废铁。

随着游戏推进，玩家会逐步写出：

拾荒脚本
自动回收循环
风险判断逻辑
资源调度系统
战斗 AI
迷宫寻路算法
多机器人任务分配系统
完整的废土自动化框架

因此，这款游戏最准确的定位不是单纯的“挂机游戏”，也不是单纯的“编程游戏”，而是：

废土机器人自动化 RPG

或者更具体地说：

废土机器人操作系统模拟器

玩家的成长不是从弱角色变成强角色，而是从“写几行简单指令”变成“搭建一套自动化生存系统”。

二、从《编程农场》学到的核心原则

《编程农场》最值得学习的地方，不是它的种田题材，而是它的阶段设计方式。

它每推进一个阶段，都会给玩家一个新机制。这个新机制不是单纯增加一种资源，而是会让玩家原来的代码失效，迫使玩家重构。

例如：

草：学习循环
树：学习空间布局
南瓜：学习全局状态检测
迷宫：学习寻路
向日葵：学习测量与最大值选择
仙人掌：学习排序
混合种植：学习局部依赖关系
巨大农场：学习多线程 / 多无人机调度
恐龙：学习自碰撞路径规划
排行榜：学习完整自动化

真正的设计原则是：

不要用“更多资源”填充内容，
而要用“会让旧代码失效的新规则”填充内容。

这对废土编程游戏非常重要。

如果只是把《编程农场》里的作物换成废土资源：

草 → 废铁
木头 → 电线
南瓜 → 高级零件
迷宫 → 废墟

那只是换皮。

更好的做法是：让每个废土机制都带来新的编程问题。

例如：

辐射区：迫使玩家写回撤逻辑
酸雨：迫使玩家写避难逻辑
敌对拾荒者：迫使玩家写避让或战斗逻辑
背包容量：迫使玩家写卸货逻辑
电池消耗：迫使玩家写回充逻辑
角斗场：迫使玩家写战斗状态机

迷宫：迫使玩家写寻路和记忆系统
多机器人：迫使玩家写调度系统

三、总体体验链路

原始链路：

捡到机器人
→ 垃圾场拾荒
→ 改装机器人
→ 角斗场战斗
→ 迷宫探险

建议扩展为完整体验链：

1. 捡到机器人
2. 修复基础行动模块
3. 垃圾场拾荒
4. 建立临时据点
5. 自动回收循环
6. 改装机器人模块
7. 进入危险垃圾区
8. 接触黑市与废土工坊
9. 解锁战斗模块
10. 角斗场试炼
11. 获得战斗许可证
12. 进入废墟迷宫
13. 找到旧世界记忆碎片
14. 修复高级 AI 核心
15. 组建多机器人小队
16. 深层迷宫与竞技联赛
17. 发现旧设施真相
18. 争夺城市核心
19. 建立自主机械聚落
20. 完全自动化 / 排行榜挑战

这条链路要同时满足三条成长线：

剧情线：

从捡到破机器人，到发现旧世界真相，再到建立机械聚落。

玩法线：

从单步指令，到资源自动化，到战斗 AI，到寻路，到多机器人调度。

数值线：

从基础废料，到模块零件，到稀有科技，到战斗积分，到旧世界核心资源。

四、阶段一：捡到机器人

1. 剧情概念

玩家在废土垃圾场中捡到一个残破机器人。

它不是强力伙伴，而是一个接近报废的旧机器：

机械臂缺失
电池漏电
传感器失灵
记忆损坏
只会执行最基础的动作

玩家最初不是“指挥机器人”，而是在“唤醒机器人”。

2. 玩法体验

这个阶段应该非常简单。

玩家只能使用少量指令：

```
move()  
turn()  
pick()  
drop()
```

第一个目标不是效率，而是让玩家获得最基础的编程反馈：

我写了一句 `move()`
机器人真的动了一步

我写了一句 `pick()`
机器人真的捡起了一块废铁

这是编程游戏最重要的早期爽点。

3. 编程能力

训练内容：

顺序执行
单步命令
基本动作反馈

4. 设计重点

这一阶段要建立情感连接。

机器人可以用非常简单的日志回应玩家：

```
> motor error...  
> command accepted.  
> I can move.  
> scrap detected.
```

让玩家觉得它不是一个抽象单位，而是一个被自己慢慢修好的废土伙伴。

五、阶段二：修好机器人

1. 剧情概念

机器人能动了，但很多功能还坏着。

玩家必须在附近垃圾堆里捡基础材料，用来修复它。

2. 核心资源

Scrap: 废铁
Wire: 电线
Cell: 电池片
Plastic: 塑料壳

3. 修复内容

修好机械臂 → pick() 更快
修好履带 → move() 更稳定
修好电池 → 可行动距离增加
修好传感器 → 解锁 scan()

4. 编程能力

训练内容：

简单循环
简单条件判断
基础资源检测

例如：

如果前方有垃圾，就捡
否则继续移动

5. 设计重点

这个阶段的奖励不要只是数值奖励，而应该是“新能力”。

例如：

机械臂修复不是简单 +10% 拾取速度，
而是让玩家可以使用 `pick()`。

传感器修复不是简单扩大视野，
而是让玩家可以使用 `scan()`。

这会让“机器人改装”与“代码能力成长”自然绑定。

六、阶段三：垃圾场拾荒

1. 剧情概念

机器人已经能基本工作，玩家开始把它派进更大的垃圾场。

垃圾场不是静态资源田，而是废土环境：

垃圾堆每天变化
不同区域产出不同资源
部分垃圾带电
部分垃圾太重
部分区域存在辐射

2. 核心循环

玩家开始写第一个真正的自动化脚本：

扫描附近资源
选择目标
移动过去

拾取
背包满了回家
电量低了回家
卸货
重复

3. 编程能力

训练内容：

while 循环
if 条件
背包判断
电量判断
基础优先级

4. 体验目标

让玩家第一次看到：

机器人自动出门
自动捡垃圾
自动回家
自动卸货

这是游戏的第一个核心挂机爽点。

5. 设计重点

这一阶段不要过早增加太多风险。

目标是先让玩家完成一个稳定自动化循环。

七、阶段四：建立临时据点

1. 剧情概念

玩家不再只是捡垃圾，而是在垃圾场边缘建立一个小据点：

充电桩
维修台
零件箱
破旧终端
小型熔炉
模块工作台

2. 玩法变化

据点让游戏从“单机器人执行脚本”变成“资源生产系统”。

玩家开始处理：

自动卸货
自动维修
自动充电
自动合成零件
自动升级模块

3. 编程能力

训练内容：

函数封装
任务拆分
状态机雏形

玩家会自然开始写：

```
work()  
return_home()  
repair()  
charge()  
unload()
```

4. 数值设计

这一阶段可以引入第一条基础生产链：

Scrap + Wire → Simple Motor
Cell + Wire → Battery Pack
Scrap + Plastic → Armor Shell

5. 设计重点

玩家应该感受到：

我不是写了一段脚本，
而是让一个小据点开始运转。

八、阶段五：改装机器人

1. 剧情概念

玩家通过旧终端识别出机器人的型号，发现它原本可能属于旧世界工程系统。

这时玩家开始深入改装机器人。

2. 模块类型

扫描器
机械臂
履带
电池
装甲
焊枪
磁吸器
芯片插槽
导航模块
战斗模块

3. 核心设计原则

模块升级不应该只是数值成长，而应该解锁新的 API。

例如：

扫描器 → `scan_area()`
机械臂 → `pick_nearby()`
电池模块 → `get_battery()`
装甲模块 → `get_durability()`
芯片插槽 → `import / function library`
导航模块 → `remember_position()`
战斗模块 → `get_enemy_distance()`

4. 设计重点

装备升级 = 编程能力升级。

玩家不只是变强，而是能写出原来写不了的代码。

九、阶段六：危险垃圾区

1. 剧情概念

普通垃圾场资源不够用了。

玩家必须进入更危险的区域：

辐射废坑
酸雨区
旧电厂
机械坟场
无人区边界

2. 新机制

危险区资源更好，但有风险：

耐久下降
电量快速消耗
传感器误报
移动失败
敌对拾荒者袭击
噪音吸引怪物

3. 编程能力

训练内容：

风险判断
回撤逻辑
异常处理
容错

玩家需要写：

如果耐久低于 30%，回家维修
如果电量低于 20%，回家充电
如果辐射高于阈值，换区域
如果背包有稀有资源，优先返回

4. 设计重点

这一阶段要让旧脚本失效。

旧脚本是：

看到垃圾就捡

新环境要求：

判断是否值得冒险

这是从“自动化”进入“策略自动化”的关键阶段。

十、阶段七：黑市与废土工坊

1. 剧情概念

玩家接触到黑市机械师、地下工坊、废土商人。

他们提供：

交易
模块蓝图
旧世界芯片
角斗场入场资格
迷宫坐标

2. 玩法变化

玩家开始面对经济判断：

哪些资源应该保留
哪些可以交易
先买模块还是先升级基地
当前目标缺什么

3. API 解锁

可以引入：

```
get_price()
get_cost()
craft()
trade()
unlock()
```

4. 编程能力

玩家开始写自动购买逻辑：

读取下一个升级成本
判断缺什么资源

切换生产区域
资源够了自动升级

5. 数值设计

这是从固定阈值进入动态调度的阶段。

早期玩家可能写：

Scrap 小于 1000 就去捡 Scrap

中期应该变成：

读取下一个升级成本
根据缺口决定要刷什么

这会让游戏进入真正的自动经济系统。

十一、阶段八：角斗场试炼

1. 剧情概念

黑市告诉玩家：想获得高级零件和废土通行权，必须让机器人进入角斗场。

角斗场是废土秩序的一部分：

资源分配靠战斗资格
高级零件来自战败机器人
声望决定能否进入更深区域

2. 核心体验

角斗场不是手操战斗，而是战斗 AI 编程。

玩家在战斗前写策略：

如果敌人近，后退 / 格挡
如果敌人远，射击 / 接近
如果热量高，停止攻击
如果血低，修理
如果敌人护盾开启，绕后

3. 编程能力

训练内容：

状态机
优先级
目标选择
距离判断
冷却管理
敌人行为识别

4. 角斗场阶段内容

废铁木桩：学习 `attack()`
履带斗士：学习距离控制
盾牌机器人：学习绕后
远程炮台：学习掩体和冲刺
修理机器人：学习打断
多敌人战：学习目标优先级
Boss 战：学习阶段状态机
镜像战：对抗自己的旧脚本

5. 设计重点

角斗场应该是“策略代码的考试”。

垃圾场考验资源调度。

角斗场考验实时决策。

两者应该共享机器人模块，但代码结构不同。

十二、阶段九：获得战斗许可证

1. 剧情概念

通过角斗场后，玩家获得废土身份：

注册斗机师
工坊会员
地下通行证
旧设施探索权

2. 系统解锁

可以解锁：

更深迷宫坐标
高级工坊蓝图
多机器人权限
武器模块
护盾模块
战斗日志回放

3. 编程能力

这里适合解锁调试相关能力：

```
log()  
debug()  
simulate()
```

例如角斗场战斗后生成日志，玩家可以根据日志优化脚本。

4. 设计重点

战斗不是支线，而是打开世界的新钥匙。

十三、阶段十：废墟迷宫探险

1. 剧情概念

玩家进入旧世界废墟：

地下工厂
坍塌地铁
避难所
军用服务器中心
机械坟场深处

这里藏着旧世界技术，也是机器人身份之谜的来源。

2. 迷宫不应只是宝箱

迷宫中可以包含：

岔路
墙体
锁门
陷阱
电力门
敌人巡逻

稀有零件
地图碎片
限电机制

3. 编程能力

训练内容：

寻路
记忆
地图构建
路径回溯
风险收益判断

初期玩家可以写贴墙法。
中期玩家会记录走过的位置。
后期玩家会写地图和路径规划。

4. API 解锁

```
can_move()  
scan_direction()  
mark_tile()  
remember_position()  
get_map()  
find_path()  
simulate_route()
```

5. 设计重点

迷宫最好和剧情深度绑定。

垃圾场给玩家资源。
角斗场给玩家资格。
迷宫给玩家真相。

十四、阶段十一：发现旧世界设施

1. 剧情概念

玩家逐渐发现，捡到的机器人不是普通废品。

它可能是：

旧世界维护机器人
军用测试机

城市 AI 的移动终端
地下设施的钥匙

它缺失的记忆碎片散落在不同迷宫中。

2. 奖励设计

玩家获得的不只是资源，还有：

记忆碎片
旧日志
设施权限
隐藏模块
新指令集

3. 剧情奖励最好对应 API

例如：

找到旧导航核心 → 解锁 `pathfind()`
找到战术模块 → 解锁 `predict_enemy()`
找到集群协议 → 解锁 `spawn_robot()`
找到记忆芯片 → 解锁 `persistent memory`

4. 设计重点

剧情不是播片，而是让玩家获得新的编程能力。

十五、阶段十二：多机器人小队

1. 剧情概念

玩家不再只有一个机器人，而是能修复更多旧机器人，组成小队。

这些机器人可以有不同型号：

搬运型
侦察型
战斗型
维修型
采矿型
黑客型

2. 玩法变化

玩家开始分配任务：

- A 去拾荒
- B 留守基地修理
- C 去角斗场训练
- D 去迷宫探路
- E 去交易站跑商

3. 编程能力

训练内容：

并发
任务分配
资源竞争
共享状态
队列
优先级调度

4. 数值设计

多机器人会让资源增长变快，但也带来新的消耗：

电力消耗增加
维修消耗增加
路径冲突增加
模块需求增加

因此这一阶段要考验调度效率，而不是单纯给玩家更高产量。

十六、阶段十三：深层迷宫与竞技联赛

1. 剧情概念

玩家已经不是普通拾荒者，而是有名的机器人操作者。

世界开放更高级挑战：

深层迷宫
高级角斗联赛
阵营委托

黑市悬赏
旧世界核心设施

2. 综合玩法

角斗场要求：

战斗 AI
敌人识别
阶段策略

迷宫要求：

图搜索
路径规划
地图记忆
资源路线

垃圾场要求：

生产调度
多机器人分工
经济系统

3. 设计重点

玩家这时应该开始写“总控系统”：

如果缺芯片，派侦察机器人去电子废墟
如果缺战斗积分，派战斗机器人去角斗场
如果发现稀有迷宫，暂停低级拾荒，组织探索队

玩家不再写单个行为，而是在写废土机器人公司的运营系统。

十七、阶段十四：旧核心觉醒

1. 剧情概念

玩家发现机器人的深层身份：

它曾经连接城市中央 AI
它是某个旧世界工程的维护节点
它身上有开启核心设施的密钥

其他势力也开始争夺它。

2. 终局玩法

可以进入：

防守据点
攻入核心设施
多机器人协同战斗
迷宫 + 战斗混合关卡
资源压力极高的长期挑战

3. 编程能力

训练综合能力：

战斗状态机
路径规划
资源调度
多机器人并发
异常恢复

4. 设计重点

终局不要只是数值 Boss，而应该是对玩家整个自动化系统的压力测试。

例如：

核心设施不断生成敌人
迷宫结构变化
补给有限
机器人会损坏
玩家必须写自动修复 / 自动补给 / 自动进攻系统

十八、阶段十五：自主机械聚落

1. 剧情概念

最终，玩家从“捡到一个机器人”走到“建立一个机器人聚落”。

这个聚落可以是：

废土上的自动工坊
机器人避难所
移动堡垒
旧世界设施的新管理者

2. 终局体验

玩家追求：

自动生产
自动战斗
自动探索
自动修复
自动升级
自动扩张

3. 排行榜 / 挑战模式

终局可以开放：

从零启动挑战
最短时间建立机械聚落
最短时间打通核心设施
自动化完整度排行
代码效率排行

4. 设计重点

玩家的目标从“通关”变成：

写一套能从零自我发展的废土机器人系统。

这会自然形成长期优化空间。

十九、三条主线结构

这款游戏可以有三个主线并行推进。

1. 机器人身份线

破损废品
→ 可行动工具

- 有残存记忆
- 旧世界设施钥匙
- 中央系统节点
- 新机械聚落核心

作用：提供剧情动机。

2. 玩家能力线

- 手动控制
 - 写脚本
 - 写函数
 - 写模块
 - 写自动经济系统
 - 写战斗 AI
 - 写探索系统
 - 写多机器人调度
 - 写完整自动化系统

作用：提供编程成长。

3. 世界探索线

- 垃圾场边缘
 - 危险废区
 - 黑市
 - 角斗场
 - 废墟迷宫
 - 深层设施
 - 旧城市核心
 - 机械聚落

作用：提供 RPG 世界感。

三条线应该在关键节点交汇：

- 修复机器人 → 解锁新 API
- 赢下角斗场 → 获得迷宫权限
- 迷宫发现记忆 → 解锁高级模块
- 多机器人调度 → 推动据点成长
- 旧核心觉醒 → 进入终局自动化

二十、资源系统设计

资源不要只是数量不同，而要分层。

1. 基础资源

Scrap
Wire
Plastic
Oil
Cell

用途：

修理
建造
基础升级
基础模块制造

2. 技术资源

Chip
Circuit
Sensor
Core
Memory Fragment

用途：

解锁 API
解锁模块
解锁自动化能力
推进机器人身份线

3. 战斗资源

Armor Plate
Weapon Part
Servo
Heat Sink
Ammo
Arena Token

用途：

角斗场装备
战斗模块

防御模块
战斗许可证

4. 探索资源

Map Data
Key Fragment
Relic
Signal
Old Core

用途：

迷宫探索
旧世界设施解锁
剧情推进
高级科技

5. 流动资源

Energy
Battery
Heat
Durability
Noise
Radiation

这些不适合做普通库存。
它们的作用是制造策略条件。

例如：

Energy 低 → 回充
Heat 高 → 停止攻击
Durability 低 → 回家修理
Noise 高 → 吸引敌人
Radiation 高 → 撤离危险区

这一类资源最重要的是让玩家写判断逻辑，而不是让玩家堆数值。

二十一、数值设计原则

1. 用资源链制造隐性瓶颈

高级目标应该依赖低级资源链。

例如：

高级芯片来自危险区
危险区需要装甲
装甲需要 Scrap 和 Plate
Plate 需要熔炉
熔炉需要能源系统

这样玩家不能只刷一个资源，而要维护完整生产链。

2. 用风险收益推动代码重构

普通垃圾场可以无脑捡。

危险垃圾区必须判断风险。

这样旧脚本会失效，玩家必须重构。

3. 用模块解锁 API

模块不只是数值升级，而是表达能力升级。

扫描器升级 → `scan_area()`
导航模块 → `remember_position()`
战斗模块 → `get_enemy()`
记忆芯片 → `dictionary / persistent memory`
调度核心 → `spawn_robot()`

4. 用滞回区间代替单一阈值

不要让资源调度使用单一阈值：

资源 < 1000 → 刷
资源 >= 1000 → 不刷

这种方式会导致抖动。

更好的方式是：

低于低线 → 进入恢复模式
高于高线 → 退出恢复模式

例如：

电池低于 20% → 回家充电
电池高于 80% → 出门工作

这会让自动化更稳定。

5. 对流动资源使用节流

Energy、Heat、Noise 这类资源不能无限追。

如果设计成：

Energy < 目标 → 一直充电

可能会导致机器人一直不工作。

应该设计为：

低于危险线才处理
处理到安全线就退出
或者只在任务需要时短暂补充

二十二、内容填充公式

每个新内容都可以按这个公式设计：

新内容 = 新资源 + 新约束 + 新 API + 新失败方式 + 新代码结构

例如：

辐射区

新资源：稀有芯片
新约束：辐射持续损伤
新 API：get_radiation()
新失败：机器人损坏
新代码结构：风险评估 / 回撤逻辑

角斗场

新资源：战斗积分
新约束：敌人会动、会攻击
新 API：get_enemy_distance()
新失败：被打爆
新代码结构：战斗状态机

迷宫

新资源：旧世界遗物
新约束：墙、陷阱、电量限制
新 API：scan_direction() / mark_tile()
新失败：迷路 / 没电
新代码结构：路径记忆

黑市

新资源：信用点
新约束：价格波动
新 API：get_price()
新失败：买贵 / 断供
新代码结构：动态交易策略

二十三、角斗场设计建议

角斗场应该是中期最重要的横向系统之一。

它的作用不是简单提供战斗，而是：

检验玩家的战斗 AI
提供高级模块
提供废土身份
解锁更深探索

角斗场的体验循环

观察敌人
→ 写战斗逻辑
→ 进入战斗
→ 查看战斗日志
→ 调整代码
→ 再战斗

战斗 API

```
get_enemy_distance()  
get_enemy_hp()  
get_self_hp()  
get_heat()  
get_ammo()  
attack()  
block()  
dash()  
repair()  
use_module()
```

战斗逻辑示例

如果血量低，后退并修理
如果敌人远，使用远程攻击
如果敌人近，格挡或闪避
如果热量高，暂停攻击
如果敌人护盾开启，绕后

敌人设计

敌人应该逐步教会玩家写更复杂代码：

近战机器人：教距离判断
盾牌机器人：教绕后
远程炮台：教冲刺和掩体
修理机器人：教打断
召唤机器人：教目标优先级
Boss：教阶段状态机

二十四、迷宫设计建议

迷宫不要只是宝箱系统，而应该是探索系统。

迷宫内容

墙
门
陷阱
钥匙
巡逻敌人
能源节点
地图碎片

旧世界日志
稀有模块

迷宫阶段

浅层：贴墙法即可通过
中层：需要记录走过的位置
深层：需要构建地图
核心层：需要路径规划与风险收益判断

迷宫 API

```
can_move()
scan_front()
scan_left()
scan_right()
mark_tile()
remember_position()
get_map()
find_path()
```

设计重点

迷宫对应的编程主题应该是：

寻路
记忆
地图
风险收益

不要和角斗场重复。

二十五、多机器人设计建议

多机器人系统应该是中后期的核心跃迁。

玩法变化

从：

一个机器人做所有事

变成：

多个机器人分工协作

机器人类型

搬运型：负责拾荒与运输
侦察型：负责扫描与开图
战斗型：负责角斗场和护卫
维修型：负责基地与队友维修
黑客型：负责旧设施门禁和迷宫机关

编程主题

```
spawn_robot()  
assign_task()  
wait_for()  
broadcast()  
shared_memory  
task_queue
```

设计重点

多机器人不是单纯提高产量，而是引入调度问题：

谁去哪里？
谁优先拿资源？
谁负责回家维修？
谁留守基地？
谁探索迷宫？

二十六、剧情与系统结合方式

剧情奖励最好不要只是文本，而应该对应系统能力。

例如：

找到旧导航核心 → 解锁路径规划 API
找到战术记录器 → 解锁战斗日志
找到记忆芯片 → 解锁持久化存储
找到集群协议 → 解锁多机器人调度
找到中央核心碎片 → 解锁终局自动化

这样剧情会变成玩家代码能力的一部分。

二十七、推荐的完整阶段路线

第一阶段：捡到机器人

目标：让机器人能动、能捡、能回家

第二阶段：基础修复

目标：修好机械臂、电池、履带、传感器

第三阶段：垃圾场拾荒

目标：写出第一个自动拾荒循环

第四阶段：临时据点

目标：自动卸货、维修、充电、合成

第五阶段：模块改装

目标：通过模块解锁新 API

第六阶段：危险垃圾区

目标：风险判断、回撤逻辑、异常处理

第七阶段：黑市 / 工坊

目标：交易、蓝图、自动升级、动态成本判断

第八阶段：角斗场试炼

目标：战斗 AI、状态机、敌人识别

第九阶段：战斗许可证

目标：解锁更深区域和迷宫权限

第十阶段：废墟迷宫

目标：寻路、记忆、地图构建

第十一阶段：旧世界设施

目标：机器人身份线推进，新 API 解锁

第十二阶段：多机器人小队

目标：并发调度、任务分配、资源竞争

第十三阶段：深层迷宫 / 竞技联赛

目标：综合算法挑战

第十四阶段：旧核心觉醒

目标：终局压力测试

第十五阶段：自主机械聚落

目标：完全自动化、排行榜、从零启动挑战

二十八、最终设计总结

这款游戏最应该抓住的核心不是“废土拾荒”，也不是“机器人战斗”，而是：

玩家通过代码，把一个破机器人训练成一个能在废土中自我生存、自我升级、自我战斗、自我探索的系统。

从《编程农场》中最值得继承的设计思想是：

每个新阶段都要引入新的编程问题。

对你的游戏来说，可以具体化为：

垃圾场：资源调度问题
危险区：风险判断问题
据点：生产链问题
改装：API 解锁问题
黑市：动态经济问题
角斗场：战斗状态机问题
迷宫：路径规划问题
多机器人：并发调度问题
旧核心：综合自动化问题
排行榜：效率极限问题

最重要的一句话：

不要用“更多资源”填充游戏，
要用“更多会让旧代码失效的新规则”填充游戏。

如果这条原则成立，你的废土挂机编程游戏就不会只是一个“编程农场换皮”，而会成为一个有 RPG 包装、有情感对象、有系统深度、有长期自动化目标的废土机器人编程游戏。